RDFIA 1 - e

Julien Kordes

October 2024

Table des matières

1 Introduction **2**

2 General architecture of the ViT model **2**

2.1 Linear Projection of flattened patches 2
2.2 MLP 2
2.3 Self-attention 2
2.4 Multi-head self-attention 3
2.5 Transformer block 3
2.6 Full ViT model 3

3 Experiment on MNIST **4**

4 Larger transformers **6**

1 Introduction

This practical focuses on a Vision transformer model called the ViT model. It describes all the different parts of its architecture and makes it possible for us to understand all the steps required to be able to classify images. It then emphasis the difficulty of tuning those models depending on the dataset we are using and how large it is.

2 General architecture of the ViT model

2.1 Linear Projection of flattened patches

To play the role of the Linear Projection, our conv2D layer has to have a stride the same as the patch size (which is also the kernel size). Moreover, we use permutations and reshape on x to get the desired output size $(batchsize, nb_{tokens}, embeddingdim)$ with $nb_{tokens} = Height * Width$.

2.2 MLP

The architecture of our MLP is pretty common. and uses a GELU activation function in stead of ReLU. Indeed, GELU applies a smoother, non-linear activation based on the Gaussian cumulative distribution, which leads to gradual transitions rather than the sharp cutoffs that ReLU introduces.

2.3 Self-attention

Unlike convolutional layers that focus on local neighborhoods around each input position, self-attention can evaluate all positions throughout the entire sequence (or image), creating a global receptive field.

However, a major challenge with self-attention is its computational and memory complexity, especially the quadratic growth relative to the sequence length. In self-attention, every element in the input sequence interacts with all others, resulting in a time and space complexity of $O(n^2)$, where n represents the length of the input.

According to the paper, the equations for a single head self-attention are:

$$SA(x) = Attention * V$$
$$Attention = \text{Softmax}(qk^T)$$

$$[q, k, v] = xU_{qkv} \text{ with } U_{qkv} \in R^{Dx3D_h}$$

2.4 Multi-head self-attention

The equations are similar but we introduce D_h the number of heads that appears in the equation:

$$Attention = \text{Softmax}(\frac{qk^T}{\sqrt{D_h}})$$

2.5 Transformer block

The equations of the Transformer block follow this pattern (x represents the embedded patches) :

- $x_1 = Norm(x)$ first we normalize the data
- $x_1 = Multihead(x_1)$ then we apply the multihead attention to the normalized data
- An other normalization is applied to the embedded patches before using the MLP : $x_2 = MLP(Norm(x))$
- Finally we add the results and also add the embedded patches $x = x_1 + x_2 + x$

2.6 Full ViT model

The class token in the Vision Transformer model is a learnable token that serves as a representative summary of the entire input image. It facilitates the classification task by aggregating information from all patches through self-attention, allowing the model to effectively learn and predict class labels. The use of a class token simplifies the design of the ViT model, making it more efficient for image classification tasks compared to traditional approaches that rely on pooling mechanisms.

Positional embeddings are numerical representations added to the input tokens. They provide information about the position of each token in the sequence, as transformers process data in a non-sequential manner.

Positional embeddings serve to encode the order of tokens, enabling the model to capture sequence information and maintain contextual relationships between tokens. By providing positional context, they enhance the self-attention mechanism, allowing the model to weigh the importance of tokens based on both their content and their positions.

3 Experiment on MNIST

- Embedding dim

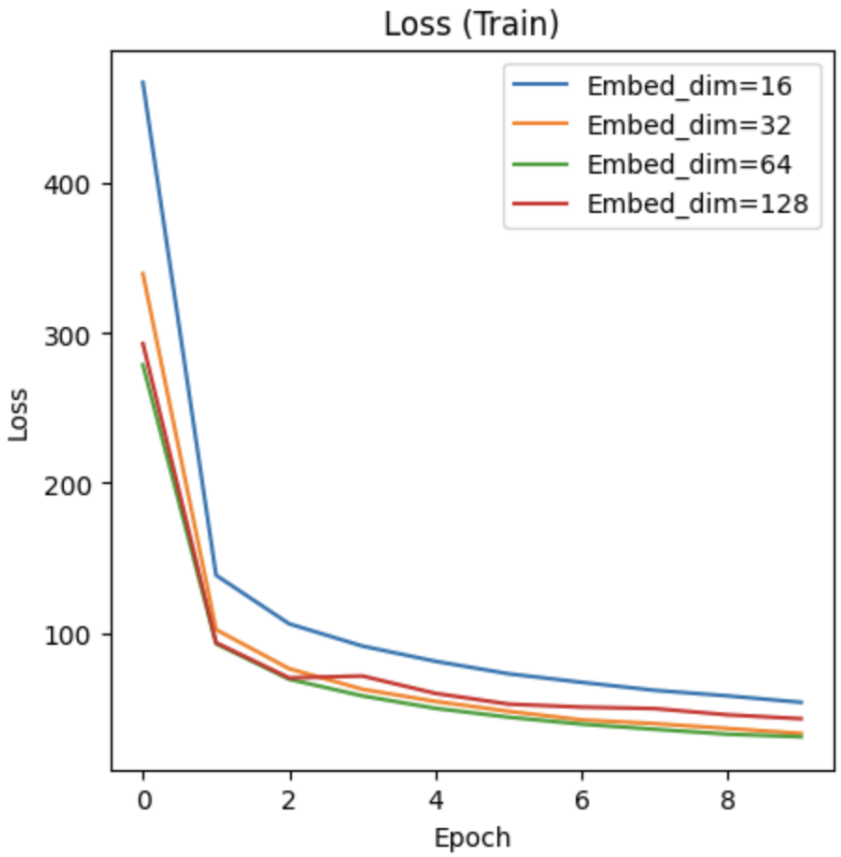

Figure 1: Influence of embedding dim on the training loss

As we can see on the FIGURE 1, the optimal embedding dimension seems to be 64. However, it doesn't seem to change that much the performance of the model since all learning curves converge pretty quickly. Moreover, the accuracy also stays close (from 96.39 for $embeddim = 16$ to **97.36** for $embeddim = 128$ after 10 epochs) The biggest difference relies in the computational resources allowed for the training : an embedding dimension of 128 requires way more parameters and therefor way more time to run but doesn't seem to give better results.

- Patch size

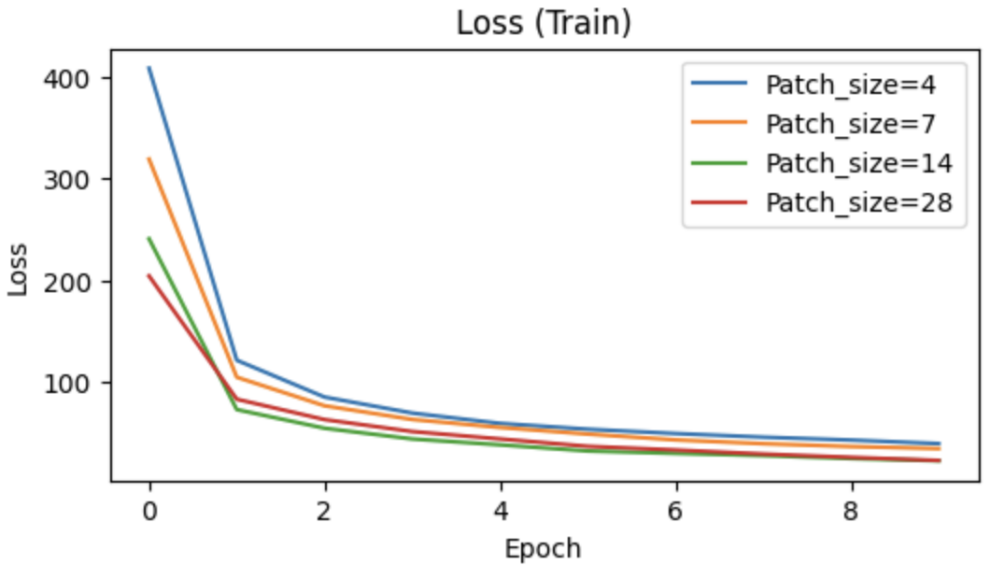

Figure 2: Influence of patch size on the training loss

This time, the computational resources increased when the patch size decreased. Indeed, a patch size of 4 led to many calculations where a patch size of 28 is equivalent to the size of a MNIST image and therefore took way less time to run. The FIGURE 2 shows that the model's performances are also pretty similar even though a higher patch size (14 or 28) seems to be more efficient. Here the highest accuracy was **97.49** obtained for patchsize = 14 and the lowest was 96.87 obtained for patchsize = 28

- Number of blocks

  Once again the model is able to get great performance and the loss converges pretty quickly. However, using a single block seems to be less efficient since it got the worse accuracy since the beginning of the tests : 93.25. The optimal performance was reached for Nb-blocks = 10 where we had an accuracy of 97.92% but it stays really close from the results obtained with 5 and even 2 blocks.

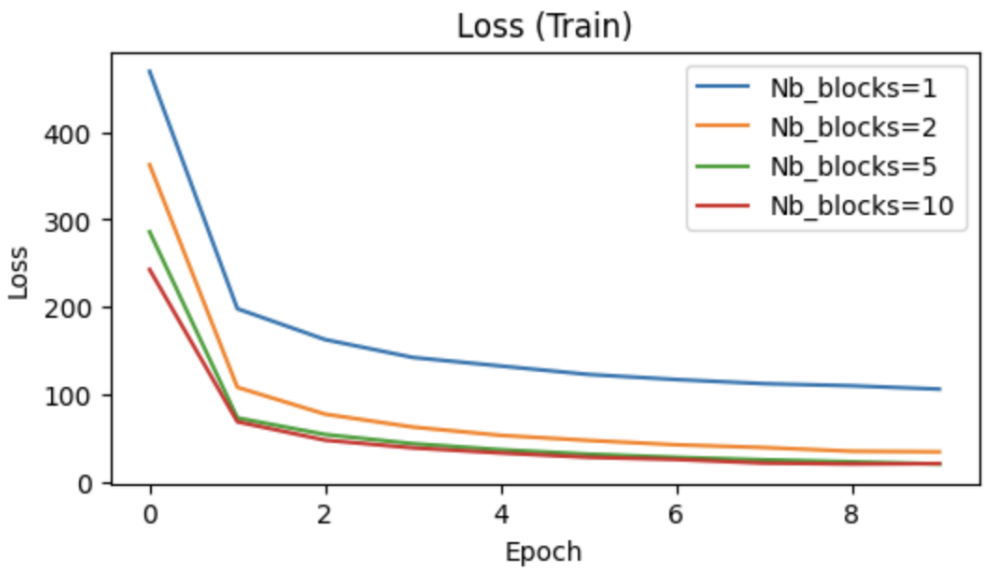

Figure 3: Influence of nb blocks on the training loss

The overall performance of the model is really good. It converges fast to a high accuracy of around 97%. The parameters that we studied can help getting a slightly faster convergence and maybe gain a little bit of accuracy but this parameter tuning doesn't change much. To try to improve this model, we could try data augmentation in order to get more information from this small dataset. Moreover using a little bit more epochs could have led to a slightly better accuracy but training those models took some time when using only a CPU and looking at the training loss, only 10 epochs looks reasonable as it seems to converge really quickly.

4 Larger transformers

The $vit - base - patch16 - 224$ model is designed and trained on 224×224 RGB images (from the ImageNet dataset). In the ViT's first layer, the input image is split into patches, which are then embedded as vectors through linear projection. That is why the model is expecting images resized to the same 224×224 dimensions which is not the case for a MNIST image (size 28×28).

However, a CNN can handle images of varying sizes until they reach the fully-connected layer.

I used the same number of epochs to train the pretrained ViT-S model to compare

its performance to the one that we implemented.

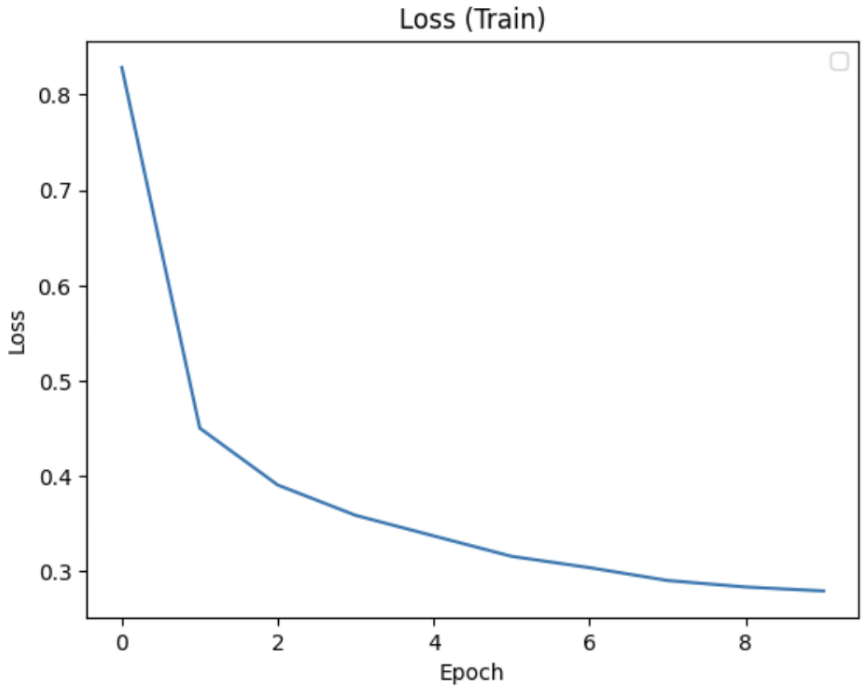

Figure 4: Training loss of the pretrained model

As we can see on the FIGURE 4, the training loss looks pretty similar to the one of the model we implemented, however the accuracy we obtained was the lowest one : only **90.13%** after 10 epochs. This might be explained by the fact that these kind of pretrained model are trained on large dataset so it's hard for them to adapt to smaller datasets like MNIST.

Training the last model which was pretrained on the ImageNet dataset was really heavy in terms of computation. It took about 20 times more time to train than the previous one (around 10 hours). Moreover the quality of the model wasn't really high This can be explained by the fact that the model is pretrained on images with big resolution and resizing the MNIST images creates a distortion in them which affects the model's performances. The issue related to the size of the data used for pretraining the model still exists and the mismatch between images with many details (used for pretraining) and images with no detail and no color like MNIST can also affect the performances.

As we can see on the FIGURE 5, the loss oscillate after the third epoch. Indeed,

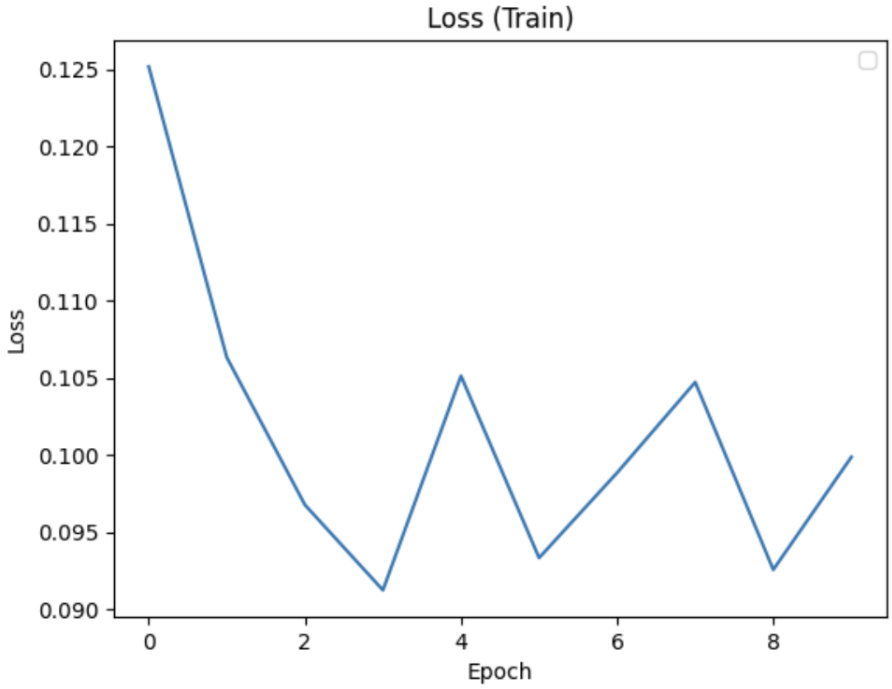

Figure 5: Training loss of the pretrained model on ImageNET

thanks to the pretraining the model was able to reach a high accuracy (96.23%) since the first epoch but wasn't able to increase it because of the problems we listed on the previous paragraph.

The paper https://arxiv.org/pdf/2012.15355v4 untitled **"Optimizing Deeper Transformers on Small Datasets"** mentions an approach called "DT-Fixup" that helps stabilizing the fine-tuning by using data-specific initializations, enabling deeper transformers to perform well on complex tasks even with smaller datasets.

Another paper called **"Efficient Training of Visual Transformers with Small Datasets"** presents multi-task learning, where the model is trained on related tasks simultaneously (like classification and position prediction), which enhances model robustness and generalization and is particularly helpful with limited data. Moreover, incorporating convolutional layers captures local relationships within data, allowing for smaller models to achieve high performance on small datasets.